The implementation of ATDD and BDD from Testing Perspectives

IOP Conf. Series: Journal of Physics: Conf. Series **1175** (2019) 012112 doi:10.1088/1742-6596/1175/1/012112

The implementation of ATDD and BDD from Testing Perspectives

Arlinta Christy Barus
Del Institute of Technology, Kab Tobasa, North Sumatera, Indonesia

Abstract. Software testing is one of the stages in software development that aims to ensure that the software is built to meet the specifications. Selection of selective test cases has a great chance of finding failure. Black box testing approach is done based on the requirement specification where this approach does not pay attention to the program code but the specification of a software. This approach can be used for testing a software using ATDD (Acceptance Test Driven Development) and BDD (Behavior Driven Development) methods. ATDD is a method of building a software created based on agile principle, acceptance test created by customer, developer and tester. BDD is a growing agile development approach in recent years. Behavior Driven Development (BDD) is built on Test-Driven Development (TDD). This Final Project focuses on the application of ATDD and BDD methods to testing the final three projects of IT Del students tested using the Roboy Framework and Cucumber Framework to find out whether these two methods are effective and be able to improve the software development process. The results obtained are that the application of ATDD and BDD methods are effective and helping to remove the error as soon as possible.

1. Introduction

Software testing is verification and validation process to ensure that software under test is working according to its requirements [11]. Some studies show that 25% to 90% of software development budget is required due to ineffective and inefficient testing phase [1]. By conducting proper testing, we can reduce the cost of software development.

Automation testing is a way of reducing cost in the software development. It is an approach of reducing human intervention in conducting testing so that the number of possible human errors and resource consumption can be reduced by using some automation tools[4]. Some automation tools that are currently widely used are such as Robot Framework, Cucumber Framework, and Selenium [5].

This paper discuses about testing two web applications developed by students of IT Del. The two applications are developed using Acceptance Test Driven Development (ATDD) and Behavior Driven Development (BDD). The objective of the study is to apply the concept of ATDD and BDD by using two tools, Robot Framework dan Cucumber Framework.

Section 2 discusses about the relevant study literature, and followed by analysis in Section 3, and then Section 4 and Section 5 contain the design and the implementation of the concept correspondingly. Conclusion and suggestion are given in the last two sections.

2. Literature Study

This section gives foundation about software testing, automation testing, Robot Framework, and Cucumber Framework.

2.1. Software Testing

Software testing is an approach to reveal errors in software. Without testing, the chance of software having big failures is higher[10]. Software testing is an essential element in software lifecycle that would be able to reduce the cost in software development and increase customer satisfaction when using the software under test[11]. Recently the software complexity increases significantly. Therefore, doing manual testing is subject to avoid including generating and executing selected test cases. [12].

Main objectives of software testing are followings: [13]:

1. Maintain the software quality
   Quality is the conformace with the pre-defined standard. One way to increase the quality is by eliminating all existing errors in the software under test. For critical software that closely related with safety and huma lives, the software quality is intolerable.
2. Validation and verifitcation
   Validation can be achieved by using some essential test cases that are able to reveal the failures of the software under test. Whilest verification is a process to ensure that such software has been developed in a proper way.
3. Reliablity Estimation

Statistical software testing gives the data of how likely the software fails. It is expected that such software is more reliable and require cost less that developing software without testing, as at the end the latent error cause higher cost to remove them from the system.

There are several ways of having software testing applied in software life cyle. One of them is V-Model that put more attention on having testing in the software life cycle, as given in Table-1 below[11].

Table 1. V-Model *Test*ing

Fase	Document	Type Test
Development Phase	*Technical Design*	*Unit Testing*
System and Integration Phase	*Functional Design*	*System Testing & Integration testing*
User Acceptance Phase	*Business Requirements*	*User Acceptance Testing*
Implementation Phase	*Business Case*	*Product Verification Testing*

To reveal errors, the seletion of test cases beomes an essential step. There are three different approaches in selecting test cases such as white box testing, black box testing, and fault based testing.

2.2 Manual and Automation Testing

Testing can be done either manually or automatically. Followings are some information about both of them.

2.2.1 Manual *Test*ing

Testing is called manual is the process does not involve any tools or scripts. In this approach, tester has a very important role to prepare, execute, and analyse the testing results. Some phases in such testing are unit testing, system testing, and user acceptance testing.

2.2.2 Automation *Test*ing

Auotomation testing is the best way to increase the effectivity, efficiency, and scope of software testing [19]. It is known as the process when testers write scripts and tools to the testing. Automation testing is urgently required when software under test has following specification:

1. Include in a big and critical project;
2. Have *Stabil Requirement*s;
3. Be accessed by virtual user;
4. Have time constraint.

Some benefits of applying automation testing are followings[19]:

1. Saving cost;
2. Increasing *Test* Coverage;
3. Reducing human errors in testing;
4. Improving software quality.

2.3 Software Development Approaches

There are various approaches in software development such as Acceptance Test Driven Development (ATDD) and Behavour Driven Development (BDD).

2.3.1 Acceptance *Test* Driven Development (ATDD)

ATDD is an approach of software development that is based on agile principle where acceptance test invove customers, developers, and testers[15]. ATDD focuses on the software requirements and functional specification of the software under test. Accordingly the implementation can be controlled well according to the specification [15].

Acceptance Test-Driven Development (ATDD) is not a testing direct approach but it is more a software development approach that ensures the implementation is according to the requirements expected by the users. [21].

Figure-1 gives the ATDD model gives the steps of conducting ATDD. Firstly, business analyst is required to write functional requirements or acceptance test[22]. Acceptance test describes functions of the software or the scope of the application under test. After writing acceptance test, developer writes test cases according to the expected outputs of the software. Then it is followed by the implementation and the execution of the test plan.

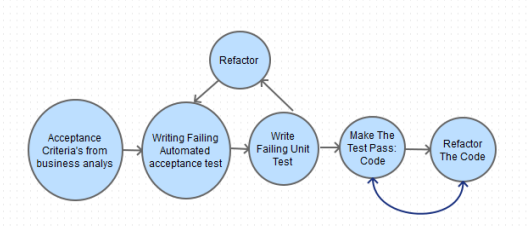

Figure 1. ATDD Model

2.3.2 Behavior Driven Development (BDD)

BDD is another agile approach that is currently widely spread of use. BDD is an expansion of Test-Driven Development (TDD). TDD is an evolusioner approach that relies on the short iteration of software development cycle and agile practice in writing code to start the automation testing before implementation, refactoring, and continuous integration[20].

There are some problems with ATDD and TDD approach that stakeholders do not understand how to do the testing as their testing technical knowledge is limited.

BDD is considered as an avolution of TDD and ATDD. BDD is focused on the behavour of the user to specify the requirments of the software under construct. BDD introduces more user friendly testing that using natural language so that all stakeholders are able to understand more about the testing process. In BDD, all test cases in acceptance test must cover all file features form various user scenarios. Each scenario is considered as a single test case based on Given-When-Then structure [23] as shown in Figure-2.

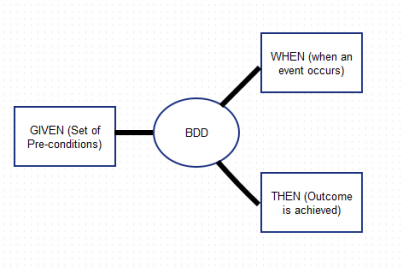

Figure 2. Given-When-Then Structure

Table 2 below gives an illustration about time and cost required to detect errors in different phases of software development.

Table 2. Cost and Time Required to Detect Errors

	Cost	**Time**				
		Requirements	**Design**	**Building**	***Testing***	**Post-release**
Time	*Requirements*	1x	3x	5-10x	10x	10-100x
	Design	-	1x	10x	15x	25-100x
	Building	-	-	1x	10x	10-25x

The data shown in Table-2 reflect the multiplication and amplified cost and time required if the we detect errors in later phases. Early detection is preferred as it spend less cost and time[24].

2.3.3 Robot Framework

Robot framework is a test automation tool that is based on the ATDD approach. Robot framework is written in Python hence it works on Python, Jython, or ironPhyton[7].

2.3.4 Cucumber

Cucumber is one of tool to do the automation testing based on the BDD approach. When using Cucumber, Cucumber will read the file specification that is known as feature, and then observe. There are some basic syntaxs to follow so that Cucumber is able to process the scenario.
If in Java coding definition phase is free of erros, then Cucumber will continue with the following step which is scenario processing. The fail or pass in the scenario phase can be monitored using Cucumber montoring fature[17].

3. Analysis

3.1 Testing analysis

3.1.1 ATTD based analysis

ATDD focuses on the requirements and functional specifications. By using ATDD, tester can control the implementation better and earlier. In ATTD, all stakeholders collaborate in defining the specification of the software. It ensures every role knows what to do and how to monitor the process.

3.1.2 BDD based analysis

BDD focused on the behavior of the user of a system under construct. BDD is quite similar to ATDD however BDD is more focused on the user friendly in understanding the BDD process. Script is writeen easily and easy to understand. In BDD, customer is directly involved in determining fiturs and requirements of the software under construct.

Followings are scenario structure of BDD as shown by Fig-3 and Fig-4.

```
Title (one line describing the story)

Narrative:
As a [role]
I want [feature]
So that [benefit]

Acceptance Criteria: (presented as Scenarios)

Scenario 1: Title
Given [context]
  And [some more context]...
When  [event]
Then  [outcome]
  And [another outcome]...

Scenario 2: ...
```

Figure 3. User Story Structure

```
Scenario 1: Title
Given [context]
  And [some more context]...
When  [event]
Then  [outcome]
  And [another outcome]...
```

Figure 4. Steps in Creating User Scenario

In ATDD some testing code can bee seen in Fig-5 as followings:

```
*** Settings ***
Library         Selenium2Library

*** Variables ***
${LoginButton.name}   name=login-button   #Login Button
${LoginUsername.id}   id=loginform-username   #Login Username Field
${LoginPassword.id}   id=loginform-password   #Login Password Field
${Login Link}   link=Login   #Login Link Text

*** Test Cases ***
TestingLoginSPI
  Open Browser   http://localhost:8080/ta%201/SYP-04(Fix)/SYP-04(Fix)/Aplikasi/advanced/frontend/web/index.php?r=site%2Flogin   firefox
  Maximize Browser Window
  Wait Until Element Is Visible   ${LoginButton.name}   30s
  Input Text   ${LoginUsername.id}   spi1
  Input Text   ${LoginPassword.id}   spi1
  Click Button   ${LoginButton.name}
```

Figure 5. Script in Login Testing using Robot Framework

In the testing execution at the beginning, there will be error messages as some of testing elements do not exist yet. Then the programmer will fix the code and re-test is conducted again. Customer involvement in ATDD in Cucumber is that in monitoring the testing result and verifiy whether programmer has fix the problem.

In BDD, developer builds the pieces of code gradually according to the expected behavior. As it is written in natural language, the process us easily followed by the stakeholders. Table 3 shows an example of Scenario login of a system under test.

Table 3. Login Feature and Skenario

Feature: To check that home page has loaded in Sistem Informasi SPI
Skenario: To check that home page has loaded
Given I am on Sistem Informasi SPI
When I click on the Login Link
And input username and password
Then I should be on the Home SPI page

In BDD, customers are more able to understand the scenario of the requirements whether in ATDD customer can see the testing output and report better.

4. Experiment Design

The section describes the experiment design in this study.

- *Study Objects*

We choose two website application development that are done by final year students of 2017 in Institut Teknologi Del. The reason is we have access to monitor the software development in this project easily,

- Experiment Stages

The procedures in conducting the experiments can be seen in Fig 6 below.

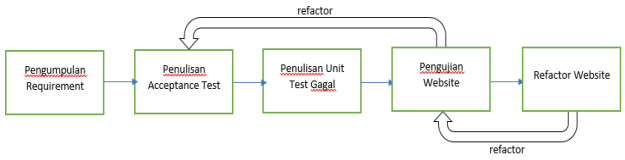

Figure 6. Experiments Phase

It starts with the requirement gathering, writing acceptance test, writing fail unit test, testing execution, and software refactoring, That will be done by using robot framework and cucumber.

5. Result

We found some bugs during the experimental phases both in BDD and ATDD approaches. It is shown that cucumber has better outputs which are report, log, and and ouput whereas robot framework is only able to create the report only.

In the first web application, using both tools, we found bugs in eight out of fifteen functions. In the second web application, we found bugs in twelve out of fifteen functions using both tools.

6. Conclusion

From the study, we can conclude that:

1. Both ATDD and BDD are approaches used to more involve all customers in the whole software life cycle.
2. Robot Framework and Cucumber Framework are automation testing tools that can apply ATDD and BDD correspondingly.
3. ATDD and BDD are useful to be used in the final year project of IT Del's students as practical exercises.
4. ATDD has complete outputs which consist of report, log, and ouput whereas robot framework is only able to create the report only.
5. In BDD, customers are more able to understand the scenario of the requirements.

References

[1] D. Firesmith, "Common *Test*ing Problems: Pitfalls to Prevent and Mitigate," Software Engineering Institute, 5 april 2013. [Online]. Available: https://insights.sei.cmu.edu/sei_blog/2013/04/common-*test*ing-problems-pitfalls-to-prevent-and-mitigate.html. [Accessed rabu 12 2016].

[2] "Why Automated *Test*ing?," Smart Bear Software, 2016. [Online]. Available: https://smartbear.com/learn/automated-*test*ing/. [Accessed Kamis desember 2016].

[3] J. Wiley and Sons, Software *Test*ing and Quality Assurance, Canada: Wiley, 2008.

[4] S. Sherry and K. Harpreet, Selenium Keyword Driven Automation *Test*ing Framework, 2014.

[5] N. v. Giessel, "Cypress - Dealing With *Test*," Xebia, 2013. [Online]. Available: http://blog.xebia.com/. [Accessed 12 Oktober 2016].

[6] "Why Selenium and Cucumber Should Not Be Used Together," *Test*ing Excellence, 25 january 2016. [Online]. Available: http://www.*test*ingexcellence.com/selenium-and-cucumber-ui-automation-challenges/. [Accessed 13 October 2016].
[7] S. Stresnjak and Z. Hocenski, "Usage of Robot Framework in Automation of Functional *Test* regression," *The Sixth International Conference on Software Engineering Advances,* 2011.
[8] "Cucumber Tutorial," W3, [Online]. Available: www.w3ii.com/id/cucumber/default.html. [Accessed 17 Oktober 2016].
[9] "Automated *test*ing with Selenium and Cucumber," [Online]. Available: https://www.ibm.com/*developer*works/library/a-automating-ria/. [Accessed 13 Oktiber 2016].
[10] R. S. Pressman, Software Engineering A practitioner's approach, New York: Higher Education, 2010.
[11] J. E. Bentley, W. Bank and C. NC, "Software *Test*ing Fundamentals—Concepts, Roles, and Terminology," vol. 30, p. 141.
[12] S. J.N, S. Thaseen.I and S. S., "Minimal *Test* Case Generation for Effective Program," *International Journal of Computer Applications (0975 – 8887),* vol. 17, 2011.
[13] S. Lumban Gaol, *Analisis Hubungan antara Test Case Behavioura,* Sitoluama, 2013.
[14] IEEE, "IEEE standard for software *test*ing document," Vols. IEEE Std 829-1998.
[15] M. Gartner, ATDD By Example, San Francisco: Addison-Wesley, 2013.
[16] A. M. Braga, D. S. Schwab and A. L. Vannucci, "The Use of Acceptance *Test*-Driven Development in the Construction of," *The Ninth International Conference on Emerging Security Information, Systems and Technologies*, 2015.
[17] M. Wynne and A. Hellesoy, The Cucumber Book, Dallas, Texas: The Pragmatic Bookshelf, 2012.
[18] R. a. T. Software *Test*ing Fundamentals—Concepts.
[19] Z. Q. Zhou, D. H. Huang, T. H. Tse, S. Yang, H. Huang and T. Y. Chen, "Metamorphic *Test*ing and Its Applications," *To appear in Proceedings of the 8th International Symposium on Future Software Technology,* vol. 12, 2004.
[20] C. Solis and X. Wang, "A Study of the Characteristics of Behaviour Driven," *EUROMICRO Conference on Software Engineering and Advanced Applications,* vol. 37, 2011.
[21] N. Koudelia, "Acceptance *Test*-Driven Development," *Master's Thesis in Information Technology,* p. 102, 2011.
[22] V. Aggarwal and M. Singh, "Acceptance *Test* Driven Development," *Journal of Advanced Computing and Communication Technologies,* 2014.
[23] M. Dienpebeck, M. Soeken, D. Grobe and R. Drechsler, "Behavior Driven Development for circuit Design and Verification".
[24] T. Point, "Behaviout Driven Development".